NEO4J AURA + GOOGLE COLAB

STEP-BY-STEP PRACTICAL TEACHING GUIDE

VERY CLEAR PLAIN PDF — NO COLOUR CODE BOXES

NEO4J AURA	Cloud graph database
GOOGLE COLAB	Python execution environment
NEO4J DRIVER	Connects Python to Neo4j
CYPHER	Language used to work with the graph

Teaching objective: Connect Google Colab to Neo4j Aura, create nodes and relationships, execute Cypher, and visually verify the graph.

1. BIG PICTURE — HOW THE SYSTEM WORKS

Google Colab → Neo4j Python Driver → Neo4j Aura

Google Colab runs Python. The Neo4j Driver sends commands to Aura. Neo4j Aura stores the graph and displays it visually.

• Aura = database.

• Colab = Python environment.

• Driver = connection bridge.

• Cypher = Neo4j query language.

Teaching method: Run code in Colab → switch to Aura → run MATCH → observe the graph.

2. GRAPH DATABASE BASICS

Concept	Meaning	Example
Node	Entity	Person: Ram
Label	Category	Person
Property	Information	name = Ram
Relationship	Connection	KNOWS
Relationship property	Information about connection	since = 2024

3. NEO4J AURA SETUP

STEP 1: Create / Open AuraDB

Open Neo4j Aura, sign in, and create an AuraDB instance. Use the available free/student option if appropriate.

Expected result: AuraDB instance is available.

Security: Keep the Aura password private. Do not publish it in a notebook or screenshot.

STEP 2: Copy Connection Details

Copy the Aura connection URI, username and password. The URI commonly begins with neo4j+s://.

STEP 3: Open Aura Query Editor

Open the Aura Query Editor. It allows Cypher commands to be executed directly in the database.

4. GOOGLE COLAB — INSTALL NEO4J DRIVER

STEP 4: Install the Python Driver

Create a new Google Colab notebook and run:

CODE / COMMAND

```
!pip install neo4j
```

Expected result: Neo4j Python package is installed.

STEP 5: Import GraphDatabase

Run:

CODE / COMMAND

```
from neo4j import GraphDatabase

print("Neo4j driver imported successfully!")
```

Expected result: Import succeeds.

5. CONNECT COLAB TO NEO4J AURA

STEP 6: Enter Credentials Securely

Use getpass so the password is not displayed.

CODE / COMMAND

```
from getpass import getpass

URI = input("Enter Neo4j Aura URI: ")
USERNAME = input("Enter username: ")
PASSWORD = getpass("Enter password: ")
```

STEP 7: Create the Driver

Create the communication object.

CODE / COMMAND

```
driver = GraphDatabase.driver(
    URI,
```

```
    auth=(USERNAME, PASSWORD)
)

print("Driver created successfully!")
```

Expected result: Driver is created.

STEP 8: Verify Connectivity

Test the connection before creating data.

CODE / COMMAND

```
driver.verify_connectivity()

print("Connected to Neo4j successfully!")
```

Expected result: Connected to Neo4j successfully.

If connection fails: Check URI, username, password, internet connection and Aura database status.

6. FIRST QUERY — HELLO NEO4J

STEP 9: Run Hello Neo4j

Python sends this Cypher query to Aura.

CODE / COMMAND

```
query = "RETURN 'Hello Neo4j' AS message"

with driver.session() as session:
    result = session.run(query)

    for record in result:
        print(record["message"])
```

Expected result: Hello Neo4j

Teaching point: Python sends the Cypher query to Neo4j Aura through the Driver.

7. CREATE THE FIRST NODES

STEP 10: Create Three Person Nodes

Create Ram, Sita and John. Each node has the label Person and a name property.

CODE / COMMAND

```
query = """
CREATE (ram:Person {name: "Ram"})
CREATE (sita:Person {name: "Sita"})
CREATE (john:Person {name: "John"})
"""

with driver.session() as session:
    session.run(query)

print("3 people created successfully!")
```

Expected result: 3 people created successfully.

Read this syntax: (ram:Person {name: "Ram"}) means variable = ram, label = Person, property = name: Ram.

STEP 11: View the Nodes in Aura

Switch to Aura Query Editor and run:

CODE / COMMAND

```
MATCH (p:Person)
RETURN p;
```

Expected result: Person nodes are displayed.

STEP 12: Display Only Names

Run:

CODE / COMMAND

```
MATCH (p:Person)
RETURN p.name AS Name;
```

Expected result: Ram, Sita and John are returned.

8. CREATE RELATIONSHIPS

STEP 13: Connect Ram to Sita and John

First find the nodes with MATCH. Then create two KNOWS relationships.

CODE / COMMAND

```
query = """
MATCH (ram:Person {name: "Ram"})
MATCH (sita:Person {name: "Sita"})
MATCH (john:Person {name: "John"})

CREATE (ram)-[:KNOWS]->(sita)
CREATE (ram)-[:KNOWS]->(john)
"""

with driver.session() as session:
    session.run(query)

print("Relationships created successfully!")
```

Expected result: Two KNOWS relationships are created.

STEP 14: Visualize the Graph in Aura

Run this query in Aura:

CODE / COMMAND

```
MATCH (p:Person)-[r:KNOWS]->(q:Person)
RETURN p, r, q;
```

Expected result: Ram is connected to Sita and John.

9. MAIN MATCH QUERY — VERY CLEAR EXPLANATION

MAIN CYPHER QUERY

```
MATCH (ram:Person)-[r:KNOWS]->(person:Person)
RETURN ram, r, person;
```

Code	Meaning
MATCH	Find a pattern
(ram:Person)	Find a Person node and call it ram
[r:KNOWS]	Find KNOWS relationship and call it r
->	Relationship direction
(person:Person)	Find connected Person node
RETURN ram, r, person	Return all three parts

Important: NEOMATCH is not a Cypher command. The correct command is MATCH.

STEP 15: Return Only Names

This is easier for beginners.

CODE / COMMAND

```
MATCH (ram:Person)-[:KNOWS]->(person:Person)
RETURN ram.name AS Person,
       person.name AS Knows;
```

Expected result: Ram | Sita and Ram | John.

STEP 16: Show Relationship Type

Use type(r).

CODE / COMMAND

```
MATCH (ram:Person)-[r]->(person:Person)
RETURN ram.name AS Person,
       type(r) AS Relationship,
       person.name AS PersonKnown;
```

Expected result: KNOWS is shown as the relationship type.

10. RUN THE SAME CYPHER FROM COLAB

STEP 17: Execute the Graph Query from Python

The same Cypher can be sent through the Python Driver.

CODE / COMMAND

```
query = """
MATCH (ram:Person)-[r:KNOWS]->(person:Person)
RETURN ram.name AS Person,
       type(r) AS Relationship,
       person.name AS Knows
"""

with driver.session() as session:
    result = session.run(query)
```

```
for record in result:
    print(
        record["Person"],
        record["Relationship"],
        record["Knows"]
    )
```

Expected result: Ram KNOWS Sita and Ram KNOWS John.

11. NODE PROPERTIES

STEP 18: Add Age and City

Use SET to add or change properties.

CODE / COMMAND

```
MATCH (p:Person {name: "Ram"})
SET p.age = 21,
    p.city = "Kakinada"
RETURN p;
```

Expected result: Ram now has age and city.

STEP 19: Read Properties

Retrieve selected properties.

CODE / COMMAND

```
MATCH (p:Person)
RETURN p.name AS Name,
       p.age AS Age,
       p.city AS City;
```

Expected result: Properties are returned.

12. RELATIONSHIP PROPERTIES

STEP 20: Add a Relationship Property

A relationship can have its own property.

CODE / COMMAND

```
MATCH (ram:Person {name:"Ram"})-[r:KNOWS]->(sita:Person {name:"Sita"})
SET r.since = 2024
RETURN ram.name, r.since, sita.name;
```

Expected result: Ram | 2024 | Sita.

13. GRAPH TRAVERSAL

STEP 21: Find People Ram Knows

Follow outgoing KNOWS relationships.

CODE / COMMAND

```
MATCH (ram:Person {name:"Ram"})-[:KNOWS]->(person:Person)
RETURN person.name AS PersonRamKnows;
```

Expected result: Sita and John.

STEP 22: Reverse Direction

Find people whose relationship points toward Ram.

CODE / COMMAND

```
MATCH (person:Person)-[:KNOWS]->(ram:Person {name:"Ram"})
RETURN person.name AS Person;
```

Expected result: Connected people are returned.

14. STUDENT–COURSE–FACULTY CASE STUDY

Graph model: Student → ENROLLED_IN → Course ← TEACHES ← Faculty

STEP 23: Create Students

Create three Student nodes.

CODE / COMMAND

```
CREATE (ravi:Student {name:"Ravi"})
CREATE (priya:Student {name:"Priya"})
CREATE (arun:Student {name:"Arun"});
```

Expected result: Three students.

STEP 24: Create Courses

Create three Course nodes.

CODE / COMMAND

```
CREATE (ds:Course {name:"Data Science"})
CREATE (dbms:Course {name:"DBMS"})
CREATE (ml:Course {name:"Machine Learning"});
```

Expected result: Three courses.

STEP 25: Create ENROLLED_IN Relationships

Connect students to courses.

CODE / COMMAND

```
MATCH (ravi:Student {name:"Ravi"})
MATCH (priya:Student {name:"Priya"})
MATCH (arun:Student {name:"Arun"})
MATCH (ds:Course {name:"Data Science"})
MATCH (dbms:Course {name:"DBMS"})
MATCH (ml:Course {name:"Machine Learning"})

CREATE (ravi)-[:ENROLLED_IN]->(ds)
CREATE (ravi)-[:ENROLLED_IN]->(dbms)
CREATE (priya)-[:ENROLLED_IN]->(ds)
CREATE (arun)-[:ENROLLED_IN]->(ml);
```

Expected result: Enrollment relationships.

STEP 26: Create Faculty Nodes

Create two Faculty nodes.

CODE / COMMAND

```
CREATE (kumar:Faculty {name:"Dr. Kumar"})
CREATE (facultyPriya:Faculty {name:"Dr. Priya"});
```

Expected result: Two faculty nodes.

STEP 27: Create TEACHES Relationships

Connect faculty to courses.

CODE / COMMAND

```
MATCH (kumar:Faculty {name:"Dr. Kumar"})
MATCH (facultyPriya:Faculty {name:"Dr. Priya"})
MATCH (ds:Course {name:"Data Science"})
MATCH (dbms:Course {name:"DBMS"})
MATCH (ml:Course {name:"Machine Learning"})

CREATE (kumar)-[:TEACHES]->(ds)
CREATE (kumar)-[:TEACHES]->(dbms)
CREATE (facultyPriya)-[:TEACHES]->(ml);
```

Expected result: Teaching relationships.

15. MULTI-HOP GRAPH QUERY

STEP 28: Find Ravi's Course and Faculty

Move from Student → Course → Faculty.

CODE / COMMAND

```
MATCH (s:Student {name:"Ravi"})
      -[:ENROLLED_IN]->
      (c:Course)
      <-[:TEACHES]-
      (f:Faculty)
RETURN s.name AS Student,
       c.name AS Course,
       f.name AS Faculty;
```

Expected result: Ravi's courses and corresponding faculty.

STEP 29: Display Complete Academic Graph

Return the connected nodes and relationships.

CODE / COMMAND

```
MATCH (s:Student)-[r1:ENROLLED_IN]->(c:Course)
MATCH (f:Faculty)-[r2:TEACHES]->(c)
RETURN s, r1, c, f, r2;
```

Expected result: Academic graph is displayed.

16. CYPHER QUICK REFERENCE

Operation	Example	Purpose
CREATE	`CREATE (p:Person {name:'A'})`	Create
MATCH	`MATCH (p:Person)`	Find
RETURN	`RETURN p`	Display
SET	`SET p.age = 20`	Add/change property
REMOVE	`REMOVE p.age`	Remove property
DELETE	`DELETE r`	Delete relationship
DETACH DELETE	`DETACH DELETE p`	Delete node + relationships
MERGE	`MERGE (p:Person {name:'A'})`	Match or create

17. MERGE

STEP 30: Use MERGE

MERGE matches an existing pattern or creates it if it does not exist.

CODE / COMMAND

```
MERGE (p:Person {name:"Ram"})
RETURN p;
```

Expected result: Existing Ram is matched; otherwise Ram is created.

18. COUNTING AND ANALYTICS

STEP 31: Count Person Nodes

Use COUNT.

CODE / COMMAND

```
MATCH (p:Person)
RETURN count(p) AS NumberOfPeople;
```

Expected result: Number of Person nodes.

STEP 32: Count Students per Course

Use COUNT and ORDER BY.

CODE / COMMAND

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
RETURN c.name AS Course,
       count(s) AS NumberOfStudents
ORDER BY NumberOfStudents DESC;
```

Expected result: Course enrollment counts.

19. SAFE DELETION

STEP 33: Delete One Relationship

Delete the selected relationship.

CODE / COMMAND

```
MATCH (ram:Person {name:"Ram"})-[r:KNOWS]->(john:Person {name:"John"})
```

```
DELETE r;
```

Expected result: Relationship is deleted.

STEP 34: Delete Node and Relationships

DETACH DELETE removes the node and its connected relationships.

CODE / COMMAND

```
MATCH (p:Person {name:"John"})
DETACH DELETE p;
```

Expected result: John and connected relationships are deleted.

> **Warning:** Always inspect the MATCH pattern before running DETACH DELETE.

20. COMPLETE GOOGLE COLAB CODE

This is the complete basic workflow. The code is deliberately presented in large black text on a white background.

COMPLETE COLAB CODE

```
# 1. INSTALL
!pip install neo4j

# 2. IMPORT
from neo4j import GraphDatabase
from getpass import getpass

# 3. ENTER AURA DETAILS
URI = input("Enter Aura URI: ")
USERNAME = input("Enter username: ")
PASSWORD = getpass("Enter password: ")

# 4. CREATE DRIVER
driver = GraphDatabase.driver(
    URI,
    auth=(USERNAME, PASSWORD)
)

# 5. VERIFY CONNECTION
driver.verify_connectivity()
print("Connected to Neo4j successfully!")

# 6. TEST CYPHER
with driver.session() as session:
    result = session.run(
        "RETURN 'Hello Neo4j' AS message"
    )

    for record in result:
        print(record["message"])

# 7. CREATE NODES
query = """
CREATE (ram:Person {name:"Ram"})
CREATE (sita:Person {name:"Sita"})
CREATE (john:Person {name:"John"})
"""

with driver.session() as session:
    session.run(query)

# 8. CREATE RELATIONSHIPS
query = """
MATCH (ram:Person {name:"Ram"})
MATCH (sita:Person {name:"Sita"})
```

```
MATCH (john:Person {name:"John"})

CREATE (ram)-[:KNOWS]->(sita)
CREATE (ram)-[:KNOWS]->(john)
"""

with driver.session() as session:
    session.run(query)

# 9. READ THE GRAPH
query = """
MATCH (ram:Person)-[r:KNOWS]->(person:Person)
RETURN ram.name AS Person,
       type(r) AS Relationship,
       person.name AS Knows
"""

with driver.session() as session:
    result = session.run(query)

    for record in result:
        print(
            record["Person"],
            record["Relationship"],
            record["Knows"]
        )
```

21. AURA + COLAB TEACHING CYCLE

Step	Action	Where
1	Write / run Python	Google Colab
2	Driver sends Cypher	Colab → Aura
3	Aura executes	Neo4j Aura
4	Data is stored	Neo4j Aura
5	Run MATCH	Aura Query Editor
6	Observe graph	Aura
7	Run same query	Colab
8	Compare results	Both

22. STUDENT PRACTICAL EXERCISE

• Create at least 5 Student nodes.

• Create at least 3 Course nodes.

• Create at least 2 Faculty nodes.

• Create ENROLLED_IN relationships.

• Create TEACHES relationships.

• Add properties to students.

• Add a property to one relationship.

• Display the complete graph in Aura.

• Find all courses taken by one student.

• Find the faculty teaching those courses.

• Use COUNT to count students in each course.

• Execute at least one Cypher query from Colab.

23. VIVA QUESTIONS

1. What is a graph database?

2. What is Neo4j Aura?

3. What is Cypher?

4. What is the Neo4j Python Driver?

5. What is a node?

6. What is a relationship?

7. What is a label?

8. What is a property?

9. What is the difference between CREATE and MATCH?

10. Why do we use MERGE?

11. What does DETACH DELETE do?

12. What is graph traversal?

13. What does -[:KNOWS]-> represent?

14. What is the purpose of r in [r:KNOWS]?

15. What is the difference between Aura Query Editor and Colab?

16. Why should the Aura password not be hard-coded?

24. FINAL RECAP

Term	Meaning
Neo4j Aura	Cloud graph database
Google Colab	Runs Python
Driver	Connects Python to Neo4j

Cypher	Graph query language
Node	Entity
Relationship	Connection
MATCH	Find a graph pattern
CREATE	Create data
RETURN	Display data
SET	Add/change properties
MERGE	Match or create
Traversal	Follow relationships

MEMORY LINE: COLAB runs Python → DRIVER connects → CYPHER travels → AURA stores/visualizes → MATCH reads the graph.